```
import tkinter as tk
from tkinter import ttk, messagebox
import pandas as pd
import shap
from lime.lime_tabular import LimeTabularExplainer
from model import train_model, predict_risk
                    # TRAIN MODEL
rf, gb, features, X_train = train_model()
                            # SHAP
shap_explainer = shap.TreeExplainer(rf)
                            # LIME
lime_explainer = LimeTabularExplainer(
    X_train.values,
    feature_names=features,
    class_names=[
        "Low Risk",
        "Moderate Risk",
        "High Risk",
        "Very High Risk"
    ],
    mode="classification"
)
                        # PREDICT FUNCTION
def predict():
    try:
        age = float(age_entry.get())
```

```
systolic = float(sys_entry.get())
diastolic = float(dia_entry.get())
body_fat = float(body_entry.get())
smoking = 1 if smoking_var.get() == "Yes" else 0
activity = 1 if activity_var.get() == "Yes" else 0
patient_data = pd.DataFrame([[
    age,
    systolic,
    diastolic,
    body_fat,
    smoking,
    activity
]], columns=features)
                        # ENSEMBLE PREDICTION
risk_class, risk_name, probability = predict_risk(
    rf,
    gb,
    patient_data
)
result_label.config(
    text=f"Predicted Risk: {risk_name}"
)
                    # SHAP EXPLANATION
shap_result = shap_explainer(patient_data)
shap_values = shap_result.values
```

```
                                # Multiclass SHAP
if len(shap_values.shape) == 3:
    shap_patient = shap_values[0, :, risk_class]
else:
    shap_patient = shap_values[0]
shap_pairs = list(
    zip(features, shap_patient)
)
shap_pairs.sort(
    key=lambda x: abs(x[1]),
    reverse=True
)
shap_text = ""
for feature, value in shap_pairs[:4]:
    if value > 0:
        direction = "increases risk"
    else:
        direction = "decreases risk"
    shap_text += (
        f"{feature}: {value:.3f} "
        f"({direction})\n"
    )
shap_box.config(
    state="normal"
)
shap_box.delete(
```

```
    "1.0",
    tk.END
)
shap_box.insert(
    tk.END,
    shap_text
)
shap_box.config(
    state="disabled"
)
                        # LIME EXPLANATION
def lime_predict(data):
    data_df = pd.DataFrame(
        data,
        columns=features
    )
    rf_prob = rf.predict_proba(data_df)
    gb_prob = gb.predict_proba(data_df)
    return (rf_prob + gb_prob) / 2
lime_result = lime_explainer.explain_instance(
    patient_data.iloc[0].values,
    lime_predict,
    labels=[risk_class],
    num_features=4
)
lime_pairs = lime_result.as_list(
```

```
    label=risk_class
)
lime_text = ""
for feature, value in lime_pairs:
    if value > 0:
        direction = "supports risk"
    else:
        direction = "reduces risk"
    lime_text += (
        f"{feature}: {value:.3f} "
        f"({direction})\n"
    )
lime_box.config(
    state="normal"
)
lime_box.delete(
    "1.0",
    tk.END
)
lime_box.insert(
    tk.END,
    lime_text
)
lime_box.config(
    state="disabled"
)
```

```
                    # PERSONALIZED RESULT
top_features = [
    feature
    for feature, value in shap_pairs[:3]
]
personalized_text = (
    f"Your predicted cardiovascular risk is: "
    f"{risk_name}\n\n"
    f"Main factors affecting this result:\n"
    f"• {top_features[0]}\n"
    f"• {top_features[1]}\n"
    f"• {top_features[2]}\n\n"
    f"Result is personalized based on the "
    f"patient input values."
)
personalized_box.config(
    state="normal"
)
personalized_box.delete(
    "1.0",
    tk.END
)
personalized_box.insert(
    tk.END,
    personalized_text
)
```

```
        personalized_box.config(
            state="disabled"
        )
    except Exception as e:
        messagebox.showerror(
            "Error",
            f"Please check the input values.\n\n{e}"
        )
                            # CLEAR FUNCTION
def clear_all():
    age_entry.delete(0, tk.END)
    sys_entry.delete(0, tk.END)
    dia_entry.delete(0, tk.END)
    body_entry.delete(0, tk.END)
    smoking_var.set("No")
    activity_var.set("Yes")
    result_label.config(
        text="Predicted Risk: -"
    )
    shap_box.config(state="normal")
    shap_box.delete("1.0", tk.END)
    shap_box.config(state="disabled")
    lime_box.config(state="normal")
    lime_box.delete("1.0", tk.END)
    lime_box.config(state="disabled")
```

```
    personalized_box.config(state="normal")
    personalized_box.delete("1.0", tk.END)
    personalized_box.config(state="disabled")
                                    # MAIN WINDOW
root = tk.Tk()
root.title(
    "Cardiovascular Risk Prediction"
)
root.geometry("1100x750")
  root.resizable(True, True)
title_label = tk.Label(
    root,
    text="Cardiovascular Risk Prediction",
    font=("Arial", 24, "bold")
)
title_label.pack(
    pady=(15, 5)
)
subtitle_label = tk.Label(
    root,
    text="Explainable Ensemble Machine Learning",
    font=("Arial", 13)
)
subtitle_label.pack(
    pady=(0, 15)
```

```
)
                # PATIENT DETAILS
patient_frame = ttk.LabelFrame(
    root,
    text="Patient Details",
    padding=15
)
patient_frame.pack(
    fill="x",
    padx=25,
    pady=10
)
ttk.Label(
    patient_frame,
    text="Age:"
).grid(
    row=0,
    column=0,
    padx=10,
    pady=8,
    sticky="w"
)
age_entry = ttk.Entry(
    patient_frame,
    width=20
)
```

```
age_entry.grid(
    row=0,
    column=1,
    padx=10,
    pady=8
)
ttk.Label(
    patient_frame,
    text="Systolic BP:"
).grid(
    row=0,
    column=2,
    padx=10,
    pady=8,
    sticky="w"
)
sys_entry = ttk.Entry(
    patient_frame,
    width=20
)
sys_entry.grid(
    row=0,
    column=3,
    padx=10,
    pady=8
)
```

```
ttk.Label(
    patient_frame,
    text="Diastolic BP:"
).grid(
    row=1,
    column=0,
    padx=10,
    pady=8,
    sticky="w"
)
dia_entry = ttk.Entry(
    patient_frame,
    width=20
)
dia_entry.grid(
    row=1,
    column=1,
    padx=10,
    pady=8
)
ttk.Label(
    patient_frame,
    text="Body Fat (%):"
).grid(
    row=1,
    column=2,
```

```
    padx=10,
    pady=8,
    sticky="w"
)
body_entry = ttk.Entry(
    patient_frame,
    width=20
)
body_entry.grid(
    row=1,
    column=3,
    padx=10,
    pady=8
)
ttk.Label(
    patient_frame,
    text="Smoking:"
).grid(
    row=2,
    column=0,
    padx=10,
    pady=8,
    sticky="w"
)
smoking_var = tk.StringVar(
    value="No"
```

```
)
smoking_combo = ttk.Combobox(
    patient_frame,
    textvariable=smoking_var,
    values=["Yes", "No"],
    state="readonly",
    width=17
)
smoking_combo.grid(
    row=2,
    column=1,
    padx=10,
    pady=8
)
ttk.Label(
    patient_frame,
    text="Physical Activity:"
).grid(
    row=2,
    column=2,
    padx=10,
    pady=8,
    sticky="w"
)
activity_var = tk.StringVar(
    value="Yes"
```

```
)

activity_combo = ttk.Combobox(
    patient_frame,
    textvariable=activity_var,
    values=["Yes", "No"],
    state="readonly",
    width=17
)

activity_combo.grid(
    row=2,
    column=3,
    padx=10,
    pady=8
)



# ========================
# BUTTONS
# ========================
button_frame = tk.Frame(root)

button_frame.pack(
    pady=10
```

```
predict_button = ttk.Button(
    button_frame,
    text="Predict Risk",
    command=predict
)
predict_button.grid(
    row=0,
    column=0,
    padx=10
)
clear_button = ttk.Button(
    button_frame,
    text="Clear",
    command=clear_all
)
clear_button.grid(
    row=0,
    column=1,
    padx=10
)
result_frame = ttk.LabelFrame(
    root,
    text="Prediction Result",
    padding=12
)
result_frame.pack(
```

```
    fill="x",
    padx=25,
    pady=8
)
result_label = tk.Label(
    result_frame,
    text="Predicted Risk: -",
    font=("Arial", 17, "bold")
)
result_label.pack(
    pady=5
)
                    # SHAP + LIME
explanation_frame = tk.Frame(root)
explanation_frame.pack(
    fill="both",
    expand=True,
    padx=25,
    pady=8
)
# SHAP
shap_frame = ttk.LabelFrame(
    explanation_frame,
    text="SHAP Explanation",
    padding=10
)
```

```
shap_frame.grid(
    row=0,
    column=0,
    sticky="nsew",
    padx=5
)
shap_box = tk.Text(
    shap_frame,
    height=7,
    width=45,
    wrap="word"
)
shap_box.pack(
    fill="both",
    expand=True
)
shap_box.config(
    state="disabled"
)
lime_frame = ttk.LabelFrame(
    explanation_frame,
    text="LIME Explanation",
    padding=10
)
lime_frame.grid(
```

```
    row=0,
    column=1,
    sticky="nsew",
    padx=5
)
lime_box = tk.Text(
    lime_frame,
    height=7,
    width=45,
    wrap="word"
)
lime_box.pack(
    fill="both",
    expand=True
)
lime_box.config(
    state="disabled"
)
explanation_frame.columnconfigure(
    0,
    weight=1
)
explanation_frame.columnconfigure(
    1,
    weight=1
)
```

```
                # PERSONALIZED RESULT
personalized_frame = ttk.LabelFrame(
    root,
    text="Personalized Result",
    padding=10
)
personalized_frame.pack(
    fill="both",
    expand=True,
    padx=25,
    pady=8
)
personalized_box = tk.Text(
    personalized_frame,
    height=5,
    wrap="word",
    font=("Arial", 11)
)
personalized_box.pack(
    fill="both",
    expand=True
)
personalized_box.config(
    state="disabled"
)
```

```
disclaimer = tk.Label(
    root,
    text="Educational prototype only - Not a medical diagnosis",
    font=("Arial", 9)
)
disclaimer.pack(
    pady=8
)
root.mainloop()
```